

Билет 5. Thread pool с блокирующей очередью. Реализация с помощью `condition_variable`

0. Формулировка билета

Thread pool с блокирующей очередью. Реализация с помощью `condition_variable`.

Главная идея билета:

Thread pool нужен, чтобы не создавать новый поток под каждую задачу или каждого клиента. Вместо этого мы заранее создаём ограниченное число worker-потоков, а задачи кладём в общую блокирующую очередь. Если задач нет, worker не крутится в busy wait, а спит на `condition_variable`.

Ключевая схема:

```
main thread / producers:
    submit(task)
    ↓
    blocking queue
    ↓
worker threads:
    pop task
    execute task
```

Для TCP-сервера:

```
main thread:
    accept(client_fd)
    pool.submit([client_fd] {
        handle_client(client_fd);
    })

workers:
    достают задачи из очереди
    выполняют handle_client(client_fd)
```

1. Зачем нужен thread pool

В прошлых билетах был подход:

один клиент → один новый `std::thread`

Пример:

```
while (true) {  
    int client_fd = accept(server_fd, nullptr, nullptr);  
  
    std::thread t(handle_client, client_fd);  
    t.detach();  
}
```

Это просто, но плохо масштабируется.

Проблемы подхода “thread per client”:

создание потока — дорого
у каждого потока есть стек
много потоков перегружают планировщик
много context switches
кэши остывают при переключениях
есть лимиты на число потоков и файловых дескрипторов
сложно корректно останавливать detached-потоки

Если у каждого потока стек порядка нескольких мегабайт, то тысячи потоков начинают требовать огромного адресного пространства и создают нагрузку на ОС.

Thread pool решает часть этих проблем:

потоки создаются заранее
их количество фиксировано
они переиспользуются
новые задачи не создают новые потоки
легче контролировать завершение

2. Главное наблюдение: M:N

Важная идея обновлённого конспекта:

Единиц конкуренции в программе может быть много, а потоков ОС должно быть мало.

Единица конкуренции — это логическая работа:

запрос клиента
задача
future-задача
файбер
корутина

Поток ОС — это дорогая единица исполнения:

стек
регистры
планирование ядром
context switch

Нам хочется:

много логических задач
↓
немного потоков ОС

Это отношение называют **M:N**:

M задач / запросов / корутин
↓
N worker-потоков

Схема:

```
task #1 ┘
task #2 ┘
task #3 ┘
task #4 ┘→ worker #1
task #5 ┘
task #6 ┘→ worker #2
task #7 ┘
task #8 ┘
```

Thread pool — первый простой шаг к такой архитектуре.

3. Thread pool как runtime

Thread pool можно воспринимать как маленький runtime внутри программы.

Он содержит:

очередь задач
worker-потоки
механизм ожидания задач
механизм остановки

Пользовательский код не должен сам создавать потоки на каждую мелкую работу. Он должен формулировать работу как задачу:

```
pool.submit([] {  
    do_work();  
});
```

А пул решает:

какой worker выполнит задачу
когда worker проснётся
как дождаться завершения потоков
что делать при остановке

Это похоже на мини-ОС внутри программы:

ОС планирует потоки на ядрах
thread pool планирует задачи на worker-потоках

4. Минимальный API thread pool

У хорошего thread pool API может быть очень маленьким.

Минимальная идея:

```
pool.submit(task);
```

или:

```
pool.enqueue(task);
```

Гарантия:

задача однажды будет выполнена одним из worker-ов

Что намеренно не обязательно включать в сам thread pool:

возврат результата
возврат ошибки
ожидание конкретной задачи
зависимости между задачами
отмена
parallel_for

Это не значит, что они не нужны. Это значит, что их можно делать отдельными инструментами:

WaitGroup – ожидание группы задач
future/promise – получение результата
cancellation token – отмена
executor – выбор места исполнения

Так API пула остаётся простым.

5. Из чего состоит thread pool

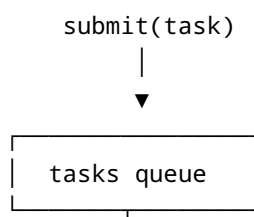
Минимальный thread pool содержит:

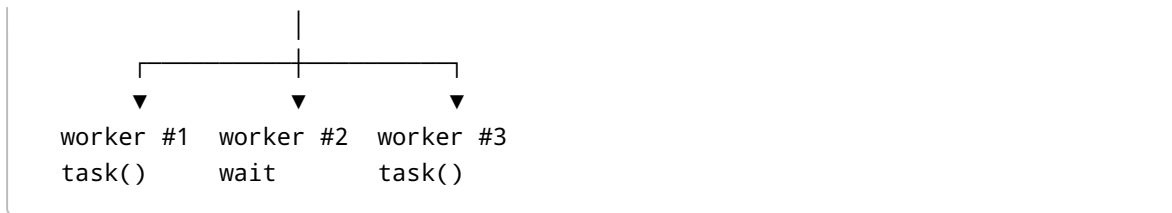
```
std::vector<std::thread> workers_;  
std::queue<std::function<void()>> tasks_  
  
std::mutex m_  
std::condition_variable cv_  
  
bool stop_ = false;
```

Смысл полей:

workers_ – заранее созданные worker-потоки
tasks_ – общая очередь задач
m_ – защищает tasks_ и stop_
cv_ – усыпляет workers, если задач нет
stop_ – флаг корректного завершения

Схема:





6. Почему нужна блокирующая очередь

Обычная очередь:

```
std::queue<std::function<void()>> tasks;
```

не потокобезопасна.

Проблемы:

```
один поток делает push
другой поток делает pop
третий поток проверяет empty
```

Если нет мьютекса, будет data race.

Но нам нужна не просто потокобезопасная очередь. Нам нужна **блокирующая очередь**:

```
push:
    положить задачу
    разбудить одного ожидающего worker-а

pop:
    если задача есть — забрать
    если задач нет — уснуть
    если очередь закрыта и пуста — вернуть false
```

Важно:

Блокирующая очередь — это не просто контейнер. Это инструмент коммуникации между потоками.

Поэтому у хорошей блокирующей очереди часто нет публичного `empty()`.

Почему?

```
if (!queue.empty()) {
    queue.pop();
}
```

В многопоточном коде это бессмысленно: между `empty()` и `pop()` другой поток может изменить очередь.

Нужна единая операция:

```
bool pop(T& out);
```

которая под мьютексом проверит условие и заберёт элемент.

7. Базовая блокирующая очередь

```
#include <condition_variable>
#include <mutex>
#include <queue>

template <typename T>
class BlockingQueue {
public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(m_);
            q_.push(std::move(value));
        }

        cv_.notify_one();
    }

    bool pop(T& out) {
        std::unique_lock<std::mutex> lock(m_);

        cv_.wait(lock, [&] {
            return closed_ || !q_.empty();
        });

        if (q_.empty()) {
            return false; // closed_ && q_.empty()
        }

        out = std::move(q_.front());
        q_.pop();

        return true;
    }
};
```

```

    }

    void close() {
        {
            std::lock_guard<std::mutex> lock(m_);
            closed_ = true;
        }

        cv_.notify_all();
    }

private:
    std::queue<T> q_;
    bool closed_ = false;

    std::mutex m_;
    std::condition_variable cv_;
};

```

Ключевые моменты:

```

q_ и closed_ защищены одним mutex
pop ждёт closed_ || !q_.empty()
push делает notify_one
close делает notify_all

```

8. Почему `pop` ждёт `closed_ || !q_.empty()`

Worker должен проснуться в двух случаях:

1. появилась задача
2. очередь закрывается

Поэтому предикат:

```
closed_ || !q_.empty()
```

Если очередь не закрыта и пуста:

```

closed_ == false
q_.empty() == true

```

worker спит.

Если появилась задача:

```
!q_.empty() == true
```

worker просыпается и забирает её.

Если очередь закрыта:

```
closed_ == true
```

worker просыпается, даже если задач нет.

После пробуждения:

```
if (q_.empty()) {  
    return false;  
}
```

Это означает:

```
очередь закрыта и задач больше нет  
worker должен завершиться
```

9. Почему нужен `while` или `wait(lock, pred)`

Нельзя писать:

```
if (q_.empty()) {  
    cv_.wait(lock);  
}
```

Проблемы:

```
spurious wakeup  
notify_all разбудил много потоков, а задача одна  
другой worker успел забрать задачу раньше
```

Поэтому:

```
cv_.wait(lock, [&] {  
    return closed_ || !q_.empty();  
});
```

эквивалентно:

```
while (!closed_ && q_.empty()) {  
    cv_.wait(lock);  
}
```

То есть после каждого пробуждения worker заново проверяет состояние.

10. Почему `notify_one` в `push`

В `push` обычно добавляется одна задача:

```
q_.push(std::move(value));
```

Значит, продолжить работу может один worker.

Поэтому:

```
cv_.notify_one();
```

Если сделать:

```
cv_.notify_all();
```

то проснутся все worker-ы, но задачу заберёт один, а остальные снова уснут.

Это лишняя нагрузка:

```
thundering herd
```

11. Почему `notify_all` в `close`

В `close` мы не добавляем одну задачу. Мы меняем состояние всей очереди:

```
closed_ = true;
```

Все worker-ы, которые спят в `pop`, должны проснуться, увидеть закрытую очередь и завершиться.

Если сделать:

```
cv_.notify_one();
```

проснётся только один worker.

Остальные могут остаться спать навсегда, а деструктор thread pool зависнет на `join`.

Поэтому:

```
cv_.notify_all();
```

12. Thread pool над блокирующей очередью

```
#include <condition_variable>
#include <functional>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

class ThreadPool {
public:
    explicit ThreadPool(size_t n) {
        for (size_t i = 0; i < n; ++i) {
            workers_.emplace_back([this] {
                worker_loop();
            });
        }
    }

    ~ThreadPool() {
        queue_.close();

        for (auto& worker : workers_) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }
};
```

```

    }

    bool submit(std::function<void()> task) {
        return queue_.push_checked(std::move(task));
    }

private:
    void worker_loop() {
        std::function<void()> task;

        while (queue_.pop(task)) {
            try {
                task();
            } catch (const std::exception& e) {
                std::cerr << "task exception: " << e.what() << "\n";
            } catch (...) {
                std::cerr << "unknown task exception\n";
            }
        }
    }

private:
    class Queue {
    public:
        bool push_checked(std::function<void()> task) {
            {
                std::lock_guard<std::mutex> lock(m_);

                if (closed_) {
                    return false;
                }

                q_.push(std::move(task));
            }

            cv_.notify_one();
            return true;
        }

        bool pop(std::function<void()>& out) {
            std::unique_lock<std::mutex> lock(m_);

            cv_.wait(lock, [&] {
                return closed_ || !q_.empty();
            });

            if (q_.empty()) {
                return false;
            }

            out = std::move(q_.front());

```

```

        q_.pop();

        return true;
    }

    void close() {
        {
            std::lock_guard<std::mutex> lock(m_);
            closed_ = true;
        }

        cv_.notify_all();
    }

private:
    std::queue<std::function<void()>> q_;
    bool closed_ = false;

    std::mutex m_;
    std::condition_variable cv_;
};

private:
    Queue queue_;
    std::vector<std::thread> workers_;
};

```

Это версия с отдельной внутренней очередью.

Идея:

ThreadPool отвечает за workers
Queue отвечает за синхронизацию producer/consumer

13. Более простой вариант кода на доске

На экзамене чаще проще написать одну структуру, без отдельного класса очереди.

```

class ThreadPool {
public:
    explicit ThreadPool(size_t n) : stop_(false) {
        for (size_t i = 0; i < n; ++i) {
            workers_.emplace_back([this] {
                worker_loop();
            });
        }
    }
};

```

```

~ThreadPool() {
    {
        std::lock_guard<std::mutex> lock(m_);
        stop_ = true;
    }

    cv_.notify_all();

    for (auto& worker : workers_) {
        if (worker.joinable()) {
            worker.join();
        }
    }
}

bool submit(std::function<void()> task) {
    {
        std::lock_guard<std::mutex> lock(m_);

        if (stop_) {
            return false;
        }

        tasks_.push(std::move(task));
    }

    cv_.notify_one();
    return true;
}

private:
void worker_loop() {
    while (true) {
        std::function<void()> task;

        {
            std::unique_lock<std::mutex> lock(m_);

            cv_.wait(lock, [this] {
                return stop_ || !tasks_.empty();
            });

            if (stop_ && tasks_.empty()) {
                return;
            }

            task = std::move(tasks_.front());
            tasks_.pop();
        }
    }
}

```

```

        try {
            task();
        } catch (...) {
            // log exception, но не даём worker-у умереть
        }
    }
}

private:
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;

    std::mutex m_;
    std::condition_variable cv_;
    bool stop_;
};

```

На экзамене этот код, скорее всего, достаточно уметь объяснить.

14. Почему задачу надо выполнять вне мьютекса

Плохой вариант:

```

std::unique_lock<std::mutex> lock(m_);

cv_.wait(lock, [&] {
    return stop_ || !tasks_.empty();
});

auto task = std::move(tasks_.front());
tasks_.pop();

task(); // плохо: выполняется под lock

```

Почему плохо?

Пока выполняется `task()`, мьютекс очереди занят.

Значит:

другие worker-ы не могут брать задачи
producer не может положить новую задачу
весь pool фактически сериализуется
одна долгая задача блокирует очередь

Правильно:

```

std::function<void()> task;

{
    std::unique_lock<std::mutex> lock(m_);

    cv_.wait(lock, [&] {
        return stop_ || !tasks_.empty();
    });

    task = std::move(tasks_.front());
    tasks_.pop();
}

task(); // вне lock

```

Критическая секция должна быть короткой:

только проверить очередь и забрать задачу

15. Корректное завершение

Thread pool должен уметь завершаться.

Шаги:

1. Запретить новые задачи.
2. Разбудить всех worker-ов.
3. Дать worker-ам доработать оставшиеся задачи.
4. Дождаться потоков через join.

В простой реализации:

```

~ThreadPool() {
    {
        std::lock_guard<std::mutex> lock(m_);
        stop_ = true;
    }

    cv_.notify_all();

    for (auto& worker : workers_) {
        worker.join();
    }
}

```


Worker выходит так:

```
if (stop_ && tasks_.empty()) {  
    return;  
}
```

Это означает graceful shutdown:

если stop == true, но задачи ещё есть – дорабатываем
если stop == true и очередь пустая – выходим

16. Почему нельзя выходить просто по stop_

Плохой код:

```
if (stop_) {  
    return;  
}
```

Что может случиться?

stop_ = true
в очереди ещё лежат задачи
worker выходит
задачи теряются

Для graceful shutdown нужно:

```
if (stop_ && tasks_.empty()) {  
    return;  
}
```

То есть:

пул останавливается
но уже принятые задачи дорабатываются

Если нужна немедленная остановка, это отдельная политика, и её надо явно описывать.

17. Почему `stop_` меняется под мьютексом

`stop_` входит в предикат ожидания:

```
stop_ || !tasks_.empty()
```

Worker читает `stop_` под мьютексом.

Значит, destructor тоже должен менять `stop_` под тем же мьютексом.

Плохо:

```
stop_ = true;  
cv_.notify_all();
```

Правильно:

```
{  
    std::lock_guard<std::mutex> lock(m_);  
    stop_ = true;  
}  
  
cv_.notify_all();
```

Иначе возможен data race.

18. Что делать с `submit` после остановки

Если `stop_ == true`, worker-ы уже завершаются или завершились.

Новая задача может никогда не выполниться.

Поэтому `submit` должен иметь понятную политику:

```
вернуть false  
или бросить исключение
```

Например:

```
bool submit(std::function<void()> task) {  
    {  
        std::lock_guard<std::mutex> lock(m_);
```

```

        if (stop_) {
            return false;
        }

        tasks_.push(std::move(task));
    }

    cv_.notify_one();
    return true;
}

```

На экзамене достаточно сказать:

После начала остановки нельзя молча принимать новые задачи, потому что их судьба становится неопределённой.

19. Exception-safety задач

Если задача бросит исключение:

```
task();
```

и это исключение выйдет наружу из функции потока, программа вызовет `std::terminate`.

Поэтому worker должен ловить исключения:

```

try {
    task();
} catch (const std::exception& e) {
    std::cerr << "task exception: " << e.what() << "\n";
} catch (...) {
    std::cerr << "unknown task exception\n";
}

```

Смысл:

одна плохая задача не должна убить worker
пул не должен деградировать из-за исключений

20. Использование thread pool в TCP-сервере

Было:

```
while (true) {
    int client_fd = accept(server_fd, nullptr, nullptr);

    std::thread t(handle_client, client_fd);
    t.detach();
}
```

Стало:

```
ThreadPool pool(4);

while (true) {
    int client_fd = accept(server_fd, nullptr, nullptr);

    bool ok = pool.submit([client_fd] {
        handle_client(client_fd);
    });

    if (!ok) {
        close(client_fd);
    }
}
```

Важно про владение `client_fd`:

ассерт возвращает fd главному потоку
главный поток передаёт fd в задачу
после submit главный поток не закрывает fd
handle_client закрывает fd

Плохо:

```
pool.submit([client_fd] {
    handle_client(client_fd);
});

close(client_fd); // плохо: worker потом получит закрытый fd
```

21. Ограничение thread pool

Thread pool ограничивает число потоков, но не делает I/O неблокирующим.

Если `handle_client` делает blocking `read`:

```
read(client_fd, buffer, sizeof(buffer));
```

и клиент молчит, worker зависает на этом клиенте.

Если в пуле 4 worker-а и 4 клиента молчат:

```
worker #1 ждёт read от client #1  
worker #2 ждёт read от client #2  
worker #3 ждёт read от client #3  
worker #4 ждёт read от client #4
```

Новые задачи будут копиться в очереди.

Вывод:

```
thread pool лучше, чем thread per client по числу потоков,  
но не решает C10k полностью.
```

Следующий шаг:

```
non-blocking I/O  
busy loop polling  
select / poll / epoll  
io_uring
```

22. Гигиена thread pool: задача не должна блокировать worker

В обновлённом конспекте это важная мысль:

Если задача блокируется, она занимает целый worker.

Для thread pool это означает:

```
worker — ценный ресурс  
блокирующая задача отнимает worker у всей программы  
если все workers заблокированы, пул остановился
```

Поэтому хороший стиль:

короткая CPU-задача – в thread pool
длгое ожидание I/O – через non-blocking I/O / epoll / io_uring / coroutine
ожидание результата – через continuation, future или scheduler, а не блокирующий get внутри worker

Иначе можно получить starvation или deadlock на уровне пула.

23. WaitGroup как отдельный инструмент

Thread pool с минимальным API не обязан уметь ждать конкретную задачу.

Но иногда надо дождаться группы задач.

Для этого можно использовать отдельный инструмент — **WaitGroup**.

Идея:

add(n) – добавили n незавершённых задач
done() – одна задача завершилась
wait() – ждать, пока счётчик станет 0

Схематичный интерфейс:

```
class WaitGroup {
public:
    void add(size_t n);
    void done();
    void wait();
};
```

Пример использования:

```
WaitGroup wg;

wg.add(items.size());

for (auto item : items) {
    pool.submit([item, &wg] {
        process(item);
        wg.done();
    });
}

wg.wait();
```

Смысл:

```
thread pool выполняет задачи
WaitGroup отвечает за ожидание группы
```

Так API пула не раздувается.

24. Возврат результата: `packaged_task` и `future`

Базовый пул выполняет задачи типа:

```
std::function<void()>
```

То есть результата нет.

Если нужен результат, можно обернуть задачу в `std::packaged_task` и вернуть `std::future`.

Пример:

```
template <typename F>
auto submit_with_future(ThreadPool& pool, F f) -> std::future<decltype(f())>
{
    using R = decltype(f());

    auto task = std::make_shared<std::packaged_task<R()>>(std::move(f));

    std::future<R> future = task->get_future();

    pool.submit([task] {
        (*task)();
    });

    return future;
}
```

Использование:

```
auto fut = submit_with_future(pool, [] {
    return 42;
});

int result = fut.get();
```

Важно:

`future/promise` — отдельная тема. В этом билете достаточно понимать, что обычный `thread pool` исполняет `void`-задачи, а результат можно добавить через `packaged_task` / `future`.

25. Единый мьютекс очереди как bottleneck

Простая реализация использует один мьютекс на всю очередь:

```
std::mutex m_;  
std::queue<std::function<void()>> tasks_;
```

При небольшой нагрузке это нормально.

Но при высокой нагрузке:

```
много producers кладут задачи  
много workers забирают задачи  
все конкурируют за один mutex
```

Очередь становится точкой сериализации.

Возможные улучшения:

```
MPMC queue с более тонкой синхронизацией  
lock-free queue  
work stealing  
несколько очередей — по одной на worker
```

26. Work stealing

Идея work stealing:

```
у каждого worker-а своя очередь задач  
если worker закончил свои задачи, он крадёт задачу из очереди другого worker-а
```

Схема:

```
worker #1: [task task task]  
worker #2: []  
worker #3: [task]
```



```
worker #2 steals from worker #1
```

Плюсы:

```
меньше contention на одной общей очереди  
лучше масштабирование  
лучше locality
```

Для экзамена по этому билету достаточно знать как развитие идеи thread pool.

27. Backpressure

Если producers создают задачи быстрее, чем workers их выполняют, очередь растёт:

```
tasks_.size() → ∞  
память заканчивается  
latency растёт
```

Backpressure — механизм, который замедляет или ограничивает producer-а.

Варианты:

```
ограниченная очередь  
семафор перед входом в систему  
отказ клиенту "server overloaded"  
лимит одновременных запросов
```

В сервере лучше обслужить ограниченное число клиентов хорошо, чем пытаться обслужить всех и деградировать.

28. Bounded vs unbounded queue

Unbounded queue

```
очередь растёт без фиксированного лимита  
push почти всегда проходит
```

Плюсы:

submit не блокируется на заполненной очереди
нет риска дедлока из-за того, что задача сабмитит подзадачи в полный queue

Минусы:

при перегрузке может расти память
нужен backpressure выше по стеку

Bounded queue

у очереди есть capacity
push ждёт или отказывает, если очередь полная

Плюсы:

ограничивает память
создаёт backpressure

Минусы:

может привести к deadlock,
если worker-задача пытается submit-ить подзадачу в полный queue

Для простого учебного thread pool обычно делают unbounded queue, а ограничение нагрузки ставят выше — например, семафором на число клиентов.

29. False sharing и cache line

В простом thread pool это не главное, но как развитие важно знать:

Если горячие поля, которые часто пишутся разными потоками, лежат в одной cache line, возникает false sharing.

Пример симптома:

worker #1 пишет counter1
worker #2 пишет counter2
counter1 и counter2 лежат в одной cache line
ядра постоянно инвалидируют cache line друг у друга

Это вызывает cache ping-pong.

Лечение:

```
struct alignas(std::hardware_destructive_interference_size) PaddedCounter {  
    std::atomic<long long> value{0};  
};
```

Полный разбор будет в теме про MESI и false sharing.

30. Размер пула

Эвристика:

CPU-bound задачи:
число worker-ов примерно равно числу аппаратных потоков

I/O-bound blocking задачи:
worker-ов иногда берут больше, потому что они часто спят

Но для сетевого сервера с большим числом медленных клиентов увеличение worker-ов — не настоящее решение.

Настоящее решение:

не блокировать worker-ы на I/O
использовать non-blocking sockets + epoll
или io_uring
или coroutine/fiber runtime

31. Типичные ошибки

Ошибка 1. Выполнять задачу под мьютексом очереди

Плохо:

```
lock(m);  
task = tasks.front();  
tasks.pop();  
task(); // под lock
```

Правильно:

```
{
    lock(m);
    task = tasks.front();
    tasks.pop();
}

task(); // вне lock
```

Ошибка 2. Использовать `if` вместо `wait(lock, pred)`

Плохо:

```
if (tasks.empty()) {
    cv.wait(lock);
}
```

Правильно:

```
cv.wait(lock, [&] {
    return stop || !tasks.empty();
});
```

Ошибка 3. В деструкторе сделать `stop = true` без `mutex`

Плохо:

```
stop = true;
cv.notify_all();
```

Правильно:

```
{
    std::lock_guard lock(m);
    stop = true;
}

cv.notify_all();
```

Ошибка 4. В деструкторе сделать `notify_one`

Плохо:

```
cv.notify_one();
```

Разбудит одного worker-а. Остальные могут остаться спать, и `join` зависнет.

Правильно:

```
cv.notify_all();
```

Ошибка 5. Выходить при `stop == true`, не проверив очередь

Плохо:

```
if (stop) {  
    return;  
}
```

Для graceful shutdown:

```
if (stop && tasks.empty()) {  
    return;  
}
```

Ошибка 6. Молча принимать задачи после остановки

Если `submit` после `stop` молча кладёт задачу, она может никогда не выполниться.

Лучше:

```
submit возвращает false  
или бросает исключение
```

Ошибка 7. Не ловить исключения в worker-е

Исключение из `task()` наружу из потока приведёт к `std::terminate`.

Ошибка 8. Закрывать `client_fd` в main thread после `submit`

Плохо:

```
pool.submit([client_fd] {
    handle_client(client_fd);
});

close(client_fd);
```

Worker получит уже закрытый fd.

Ошибка 9. Думать, что thread pool полностью решает C10k

Thread pool ограничивает число потоков, но если задачи блокируются на I/O, все worker-ы могут зависнуть.

32. Что сказать на экзамене: короткая версия

Можно отвечать так:

Thread pool нужен, чтобы не создавать поток под каждого клиента или каждую задачу. Мы заранее создаём фиксированное число worker-потоков, а задачи кладём в общую блокирующую очередь. Это реализует идею M:N: много логических задач мультиплексируются на меньшее число потоков ОС.

Очередь задач — общая структура, поэтому она защищается мьютексом. Worker в цикле пытается достать задачу. Если очередь пуста, он не крутится в busy wait, а спит на `condition_variable`. Обычно предикат ожидания такой: `stop || !tasks.empty()`.

В `submit` мы под мьютексом кладём задачу в очередь и вызываем `notify_one`, потому что появилась одна задача и достаточно разбудить одного worker-a. В деструкторе или `stop` нужно под мьютексом выставить флаг остановки и вызвать `notify_all`, чтобы проснулись все worker-ы, которые могли ждать на пустой очереди.

Worker должен выполнять задачу вне мьютекса очереди. Под lock нужно только достать задачу из очереди. Если выполнять `task()` под lock, весь пул фактически станет последовательным.

Для корректного graceful shutdown worker выходит только если `stop == true` и очередь пуста. Тогда уже принятые задачи дорабатываются, новые задачи после остановки отклоняются, а деструктор делает `join` всех worker-потоков.

В TCP-сервере главный поток после `accept` делает `pool.submit([fd] { handle_client(fd); })`. Но thread pool не решает C10k полностью: если `handle_client` блокируется на `read`, то занятый клиентом worker не может выполнять другие задачи. Поэтому следующий шаг — non-blocking I/O, `epoll` или `io_uring`.

33. Мини-код для ответа на доске

```
class ThreadPool {
public:
    explicit ThreadPool(size_t n) : stop_(false) {
        for (size_t i = 0; i < n; ++i) {
            workers_.emplace_back([this] {
                worker_loop();
            });
        }
    }

    ~ThreadPool() {
        {
            std::lock_guard<std::mutex> lock(m_);
            stop_ = true;
        }

        cv_.notify_all();

        for (auto& worker : workers_) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

    bool submit(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(m_);

            if (stop_) {
                return false;
            }

            tasks_.push(std::move(task));
        }

        cv_.notify_one();
        return true;
    }

private:
```

```

void worker_loop() {
    while (true) {
        std::function<void()> task;

        {
            std::unique_lock<std::mutex> lock(m_);

            cv_.wait(lock, [this] {
                return stop_ || !tasks_.empty();
            });

            if (stop_ && tasks_.empty()) {
                return;
            }

            task = std::move(tasks_.front());
            tasks_.pop();
        }

        try {
            task();
        } catch (...) {
            // log; worker не должен умереть
        }
    }
}

private:
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;

    std::mutex m_;
    std::condition_variable cv_;
    bool stop_;
};

```

Что объяснить:

workers_ – фиксированный набор потоков
 tasks_ – очередь задач
 m_ защищает tasks_ и stop_
 cv_ усыпляет workers, когда задач нет
 submit кладёт задачу и делает notify_one
 destructor ставит stop и делает notify_all
 worker ждёт stop || !tasks.empty()
 task выполняется вне lock

Использование в сервере:


```

ThreadPool pool(4);

while (true) {
    int client_fd = accept(server_fd, nullptr, nullptr);

    bool ok = pool.submit([client_fd] {
        handle_client(client_fd);
    });

    if (!ok) {
        close(client_fd);
    }
}

```

34. Мини-проверка

1. Зачем нужен thread pool?
2. Какую проблему подхода “thread per client” он решает?
3. Что означает идея M:N?
4. Почему thread pool можно назвать runtime внутри программы?
5. Почему API пула лучше держать минимальным?
6. Какие основные поля нужны в thread pool?
7. Почему очередь задач — это инструмент коммуникации, а не просто контейнер?
8. Почему у блокирующей очереди плохо иметь публичный `empty()`?
9. Зачем очередь задач защищать мьютексом?
10. Зачем в thread pool нужна `condition_variable`?
11. Что делает worker, если очередь пустая?
12. Почему worker ждёт `stop || !tasks.empty()`?
13. Почему нужен `while` или `cv.wait(lock, pred)`?
14. Что делает `submit`?
15. Почему в `submit` обычно `notify_one`, а не `notify_all`?
16. Почему при `close` /деструкторе нужен `notify_all`?
17. Почему задачу нужно выполнять вне мьютекса очереди?
18. Что случится, если выполнять `task()` под lock?
19. Зачем нужен `stop` или `closed`?
20. Почему `stop` меняется под тем же mutex?
21. Почему worker должен выходить только при `stop && tasks.empty()`?
22. Что плохого в `if (stop) return;`?
23. Что делать с `submit` после остановки пула?
24. Зачем в деструкторе делать `join`?
25. Почему нужно ловить исключения внутри worker-а?
26. Как использовать thread pool в TCP-сервере?
27. Кто должен закрывать `client_fd`, если задача с ним положена в пул?
28. Почему thread pool не полностью решает C10k?
29. Что будет, если все workers заблокировались на медленных клиентах?
30. Что такое WaitGroup?
31. Как получить результат задачи через `packaged_task` / `future`?
32. Почему единый mutex очереди может стать bottleneck?

- 33. Что такое work stealing?
 - 34. Что такое backpressure?
 - 35. Чем bounded queue отличается от unbounded queue?
 - 36. Почему bounded queue может привести к deadlock, если задача submit-ит подзадачи?
 - 37. Как примерно выбирать размер пула для CPU-bound задач?
 - 38. Почему для I/O-bound blocking задач thread pool — только временное решение?
-

35. Самый короткий ответ, если времени совсем мало

Thread pool — это фиксированный набор worker-потоков и общая блокирующая очередь задач. Он нужен, чтобы не создавать новый поток на каждого клиента или каждую задачу. Это реализует идею M:N: много логических задач выполняются на небольшом числе потоков ОС.

`submit` кладёт задачу в очередь под мьютексом и вызывает `notify_one`. Worker-поток ждёт на `condition_variable`, пока выполнится предикат `stop || !tasks.empty()`. Если задача появилась, worker достаёт её из очереди и выполняет вне мьютекса. Если `stop == true` и очередь пуста, worker завершает цикл.

В деструкторе thread pool под мьютексом выставляется `stop = true`, затем вызывается `notify_all`, чтобы разбудить всех спящих worker-ов, и после этого делается `join`. Важно выполнять `task()` вне lock-а и ловить исключения внутри worker-а.

В TCP-сервере главный поток после `accept` кладёт в пул задачу `[fd]` `{ handle_client(fd); }`. Но если `handle_client` блокируется на `read`, worker занят этим клиентом, поэтому thread pool не решает C10k полностью. Для большого числа медленных соединений нужны неблокирующие сокет, `epoll` или `io_uring`.